MPOO
18-Oct-2026
CORRECCION DEL EXAMEN - Esquivel Cortes Iker Alonso

PREGUNTAS ABIERTAS (0.1-0.5)

PREGUNTA 0.1:
En la radiografía de un objeto, ¿qué representa un atributo y que representa un método? Da un ejemplo de cada uno.

CORRECCIÓN:
Los atributos son cualidades o partes que forman el estado de un objeto (por ejemplo una calculadora: los botones y el cobr), y los métodos son las acciones que el objeto sabe hacer (por ejemplo, sumar o restar)

PREGUNTA 0.2:
¿QUÉ hace el método main y porqué se declara static en los programas que usamos en clase?

CORRECCIÓN:
El método main es el punto de entrada desde donde arranca la ejecución del programa. Se declara "static" para poder invocarlo directamente desde la clase sin necesidad de crear una instancia previa de la misma.

PREGUNTA 0.3:
¿Qué diferencia hay entre un atributo de instancia y uno static?

CORRECCION
Un atributo de instancia es aquel que solo le posee un objeto, y uno static pertenece a toda la clase de objeto, como ejemplo una marca de zapatos, comparten el mismo modelo pero el color puede ser diferente.

PREGUNTA 0.4:
¿Qué guarda una variable de tipo primitivo y que guarda una de referencia? Da un ejemplo de cada una.

CORRECCIÓN:
Una variable de tipo primitivo (como "int a=1;") guarda el valor directamente en la memoria. Una variable de referencia (como String o un arreglo) guarda una referencia o dirección de memoria que apunta a un objeto, no el objeto mismo. El valor null indica que la referencia no apunta a ningún objeto, pero por sí solo no define una variable de referencia.

SkyBook

MPOO 18-Oct-2026

Esquivel Cortés Iker Alonso

PREGUNTA 0.5

¿Qué hace javac con un archivo .java y que hace JVM con el resultado?

CORRECCIÓN:

javac compila el archivo .java y genera un archivo .class que contiene el bytecode. La JVM (Java Virtual Machine) es la encargada de leer e interpretar/ejecutar ese archivo .class.

R1- ¿QUE IMPRIME?

Escribe la salida exacta, una línea por renglón. El fragmento es un main válido:

```
int[] datos = {7,5,3};
int[] alias = datos;
int i=0;
while (i<3){
    System.out.println(alias[i]/2)
    i++;
}
alias = new int[]{9,8,6};
System.out.println("B"+datos[0]+alias[0]);
double mitad = (double)(datos[1]/2);
System.out.println(mitad);
```

CORRECCIÓN:

3
2
1
B79
2.0

Tabla de seguimiento:

1- while:

datos = {7,5,3} ; alias apuntan al mismo arreglo

1ra Vuelta (i=0): alias[0]	2 → 7	2 = 3 (división entera)
2da Vuelta (i=1): alias[1]	2 → 5	2 = 2.
3ra Vuelta (i=2): alias[2]	2 → 3	2 = 1.

2- Reasignación de alias: alias = new int[]{9,8,6}; cambia el apuntador de alias, pero no afecta al arreglo original (datos).

3- System.out.println("B"+datos[0]); → imprime "B" concatenando el primer elemento original de datos (7) y el de alias (9): B79

4- La última operación: double mitad = (double)(datos[1]/2); datos[1] es 5, al hacer la división entera 5/2 antes del cast o afectando directamente el casteo a double convierte el 5 a 5.0, y al dividir entre 2 da 2.0

SkyBook

MPOO

18-Oct-2026

Esquivel Cortés Iker Alonso

R2- ¿COMPILA, TRUENA O FUNCIONA?

Cada inciso va en un main independiente. Indica no compila, compila, pero truena o funciona y da la causa en una frase. Si funciona escribe la salida (separa líneas con //)

a)
```
int modo=1;
switch (modo){
    case 1: System.out.println("LISTO");
    case 2: System.out.println("FIN");
}
```

CORRECCIÓN:

CLASIFICACIÓN: Funciona.

CAUSA O SALIDA: Funciona pero al no poner "break" en "case 1:" sigue cayendo y ejecuta los demás cases de abajo; Salida: /LISTO/FIN

b)
```
{int nivel = 4;}
System.out.println(nivel);
```

CORRECCIÓN:

CLASIFICACIÓN: No compila.

CAUSA O SALIDA: Ocurre un error de ámbito (scope) la variable "int nivel = 4;" esta declarada dentro de un bloque delimitado por llaves {}, por lo que su existencia muere al cerrarse ese bloque. Al intentar usarla fuera en el System.out.println, el compilador marca error porque no lo reconoce.

c)
```
String etiqueta=null;
System.out.println(etiqueta.trim());
```

CORRECCIÓN

CLASIFICACIÓN: Compila pero truena.

CAUSA O SALIDA: La sintaxis es correcta y compila, pero en tiempo de ejecución el método .trim() sobre una variable (etiqueta) cuyo valor es null. Lanza una excepción de tipo "NullPointerException"

SkyBook

MPOO
18-Oct-2026
Esquivel Cortés Iker Alonso

R3 - MODELADO: BASE, RETO Y JUSTIFICACIÓN

Cada cargador fija su capacidad al fabricarse y nace vacío. Cuenta ranuras ocupadas y cargadores fabricados. Escribe colocar() y cuenta ranuras ocupadas y cargadores fabricados. Supón que solo se llama colocar cuando hay espacio. No uses arreglos.

BASE: Declara al menos dos atributos de instancia con tipo, uno static y uno constante final (¿también static?). Completa el constructor y dos métodos: cambio y consulta.

RETO: ¿Qué es final y qué static?: capacidad o fabricados. ¿Alguno necesita ambos?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

```
class Cargador {
    private final int capacidad;
    private int ranurasOcupadas;
    private static int fabricados=0;

    //Constructor
    public Cargador(int capacidad){
        this.capacidad=capacidad;
        this.ranurasOcupadas=0;
        fabricados++;
    }
    //Método colocar
    public boolean colocar(){
        if(ranurasOcupadas<capacidad){
            ranurasOcupadas++;
            return true;
        }
        return false;
    }
    //Método consulta de ocupadas
    public int consultarOcupadas(){
        return ranurasOcupadas;
    }

    //Método consulta de fabricados (static)
    public static int consultarFabricados(){
        return fabricados;
    }
}
```

RETO Y JUSTIFICACIONES

Capacidad: Es de instancia y final, ya que cada cargador fija su capacidad al fabricarse y NO cambia.

Fabricados: Es un atributo static, porque lleva el conteo global de todos los objetos cargadores creados en la clase. Ninguno necesita ser ambas cosas a la vez.

Constructor: Recibe la capacidad para inicializar el estado interno de cada objeto cargador y actualiza el contador global de fabricados.

Métodos: colocar valida el espacio disponible de forma mutable incrementando las ocupadas, y los métodos de consulta devuelven el estado actual sin alterar los valores.

SkyBook

R4- CÓDIGO CON REPETICIÓN:
El arreglo intentos contiene exactamente tres códigos enteros, en el orden que se probaron. Devuelve el número del primer intento que coincide con clave (1, 2 o 3). Si ninguno coincide, devuelve 0. Usa do-while y deja de revisar al acertar. El arreglo no es null, no lo modifiques.
static int intentoValido(int[] intentos, clave)
Ejemplos (argumentos → retorno):
{13,42,8}, 42→2 · {42,8,43}, 42→1, {1,2,3}, 42→0
Escribe el método completo. No imprimas ni leas del teclado.

```
static int intentoValido(int[] intentos, clave){
  int i=0;
  do{
    if(intentos[i]==clave){
      return i+1; //Devuelve la posición basada en 1(1,2,3)
    }
    i++;
  }while(i<intentos.length);
  retur 0; // Si no coincide ninguno
}
```

- Si intentos={13,42,8} y clave=42: en la posición i=1 se encuentra el 42, por lo que devuelve 2 de inmediato y se detiene gracias al do-while (segundo intento)

- Si intentos={42,8,43} y clave=42: en la posición i=0 encuentra el 42 por lo que devuelve el 1 de inmediato y se detiene gracias al do-while.

- Si intentos={1,2,3} y clave=42: recorre todo el areglo sin coincidencia el ciclo termina y devuelve 0.

SkyBook

P5 - Código con selección:

Devuelve la acción del módulo con switch sobre Modo. Primero: Si batería<10, devuelve "CARGAR" incluso con modo null. En otro caso, null devuelve "SIN MODO" antes del switch. El enum ya existe: escribe solo el método. No imprimas ni leas del teclado.

enum Modo {APAGADO, MEDIR, CALIBRAR, PRUEBA}
static String accion(Modo modo, int batería)

Regla o Valor Recibido	TEXTO DEVUELTO
APAGADO	"DETENER"
MEDIR	"LEER"
CALIBRAR	"AJUSTAR"
PRUEBA	"SIMULAR"

```
static String accion(Modo modo, int bateria){
    if(bateria<10){
        return "CARGAR";
    }
    if(modo==null){
        return "SIN MODO";
    }
    switch(modo){
        case APAGADO:
            return "DETENER";
        case MEDIR:
            return "LEER";
        case CALIBRAR:
            return "AJUSTAR";
        case PRUEBA:
            return "SIMULAR";
        default:
            return "SIN MODO";
    }
}
```

- Si la batería es menor a 10, la primera condición atrapa el caso y devuelve "CARGAR" sin importar el modo es null o cualquier otro.
- Si el modo es null, la segunda validación devuelve "sin modo" antes de evaluar el switch.
- Si el modo es APAGADO, el switch direcciona al case APAGADO y devuelve "DETENER"
- Si el modo es MEDIR, devuelve "LEER".
- Si el modo es CALIBRAR, devuelve "AJUSTAR".
- Si el modo es PRUEBA, devuelve "SIMULAR".